SmartGen: An extension to RUGRAT - Random Application Benchmark Generator

Shraddha Ghogare,Student, University of Texas at Arlington, Aditya Mhatre,Student, University of Texas at Arlington and Christoph Csallner,Associate Professor, University of Texas at Arlington

Abstract—In the world of technology, where advancements in terms of technology are ascending everyday, mobile development has gained considerable attention as well. Compared to other mobile Operating Systems, as Android OS is open source, the application generation has been on continuous increase ever since it's inception. Benchmarking is also of equal importance as every programmer has different way of writing code which leads to inconsistency to the code repository in terms of program analysis. There is also a need of a tool that can generate random application with millions of lines of code efficiently within stipulated time. Therefore, a tool RUGRAT is developed which can be used to evaluate program analysis and testing tools.
This paper extends the aforesaid tool to add Android capability in the auto-generated random codes. Abstract syntax trees have been used as the basic guideline to generate java classes, manifest XML and UI Elements generation. The generated applications have also been analyzed using FindBugs and Lint.

Index Terms—Program analysis, random, benchmarking, program generation, RUGRAT, Android, AST, UI Elements, Manifest, evaluation, bugs.

1 Introduction

In today's world of computing, mobile application have gained a lot of popularity compared to the previous era of web applications. Most of the popular tools and web applications/ web sites have their own mobile counterpart which is tremendously contributing to their revenue. On the other hand, program analysis domain has also become popular as analyzing behavior of programs such as correctness, safety and liveliness, program analysis has become crucial to the IT sectors now. [1].

Just like before buying any hardware or a computer system, benchmarking tools are now used heavily to check if that particular processor really has the capabilities providers claim to have. Similarly, programs and tools also have benchmark codes against which the capabilities of the code are tested. In order to evaluate this correctness and robustness of programs, benchmarks are used in the world of computer science. This evaluation is also applied to analysis of various algorithms and tools.

Although there exists various benchmarks that can be used to address this issue, not many of them address the custom built constraints. Moreover, as many programmers have different coding style and biases, to use a program as benchmark has become an arduous problem. Therefore, [2] developed a tool - RUGRAT which addresses a custom constrained auto-generation of programs which can be used as benchmarks.

RUGRAT tool tackles this problem by automatically creates random codes that match the custom build criteria and can be used as benchmarks to evaluate program analysis and testing tools with 10M LOC using a novel approach of parsing using stochastic trees. [2] The tool has been implemented in Java and has been tested to generate benchmarks considering multiple parameters.

However, the tool does not generate java files with third party libraries like databases, cloud APIs, Android code generation, etc. as of now. This projects aims to inculcate Android code in the RUGRAT tool. Android coding mainly revolves around java and most of the code is similar to that of basic java programming. For an instance, it uses same looping and conditional syntax as that of basic java codes. This property of Android hence made it easy to extend it to the RUGRAT tool. The project was implemented iteratively to keep adding diverse set of Android functionalists.

The implied goal of the project is also to contribute to the open source community as both Android and RUGRAT are open source.

This paper is further divided as - Section 2 delineates related work done in Random program generators, whereas Section 3 talks about working principle of RUGRAT and proposed system. It also describes various android classes and methods used. Section 4 experiments with setup and results, whereas, Section 5 delineates work that can be done in time to come. Finally, Section 6 concludes the paper.

2 Related Work

This section identifies the work done in bench-marking java tools.

Using benchmarks effectually in order to evaluate programs has always been an interesting research area in computer science and research industry. There has been tremendous work done to evaluate these benchmarks. [3] introduced the DaCapo benchmarks for Java and different parameters to consider the methodologies to choose and evaluate available benchmarks.

• *S. Ghogare and A. Mhatre was with the Department of Computer Science and Engineering, University of Texas at Arlington, TX, 76019. E-mail: shraddha.ghogare@mavs.uta.edu and aditya.mhatre@mavs.uta.edu*

To start with, SPEC benchmarks have been studied which are most widely used even today as the standard to analyze the robustness and performance of Java applications. Moreover, there are various other Java benchmarking suites available like Java Grande [4], Jolden [2], [5], and Ashes [17], which are most frequently used in the industry. Java Grande essentially works on smaller lines of code and is single threaded [11,23].

RUGRAT however asserts that, cost to develop such benchmarking tools is comparatively higher and the real-world applications need an open source, refined approach.

Along with aforesaid benchmarking tools, there has also been considerable development in the area of random program generator. For an instance, the tool Csmith is used to generate random C codes that statically and dynamically conform to the C99 standard. It not only generated random programs but also It is useful for stress-testing compilers, static analyzers, and other tools that process C code. Csmith has successfully found bugs in every tool that it has tested, and [2] has used it to find and report more than 400 previously-unknown compiler bugs. [6]

Moreover, one of the key features of Csmith was also to improve the quality of C compilers. Csmith works by creating a random C program differential test, compiles it and then executes it. The generated outputs are then compared for differential testing in order to find bugs in gcc, tcc and LLVM.

ASTGen, another re-factoring tool, as the name implies focuses on improving the written code. It is used to generate Abstract Syntax Trees that allows language and compiler developers to be used by parsers (or parser generators) to record the result of a parse phase [7].

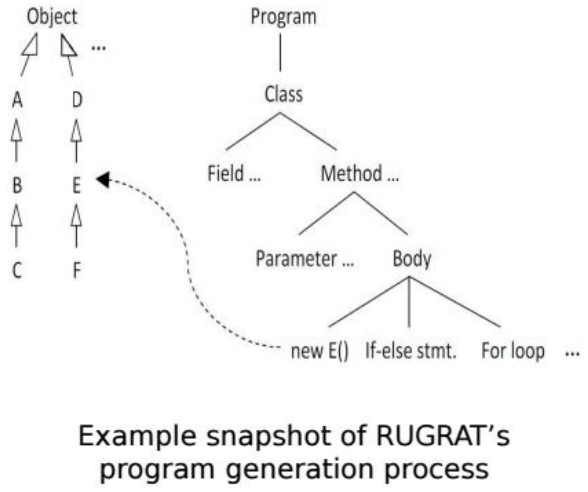

Fig. 1. Abstract Syntax Tree in RUGRAT [2]

Since benchmarks should not be too simple or too complex to work with or too costly or open source, RUGRAT was developed. RUGRAT automatically creates random applications that match the input criteria mentioned by the users. These input criteria can also be taken from a default input XML which has pre-defined software input metrics. RUGRAT has derived these metrics using an Eclipse plug-in Metrics 1.3.6. Developers are therefore given a provision to configure what properties they want in benchmark applications.

RUGRAT is scalable in the sense that it can generate very large benchmark apps (e.g., 10MLOC). RUGRAT typically works around well-formated rules. For an instance, the visibility of methods in java files is crucial as information hiding is one of the popular object oriented concept used in programming world. Moreover, multiple inheritance is not allowed practically in java and final fields should be initialized and used carefully. Referencing of non-static fields is also of great importance as it directly affects the runtime analysis of programs [2].

Along with aforesaid watch points, there should not be compiler errors or run time exceptions like divide by zero. Moreover, if a programmer uses recursion, it should be done in a controlled manner or avoided altogether in order to save heap from exhausting. [2] considers all such issues and has developed the tool that tackles these problems both compile time and run time. The tool uses tables and maps to solve these concerns in an effectual manner.

This paper extends RUGRAT to add Android capability in the randomly generated applications.

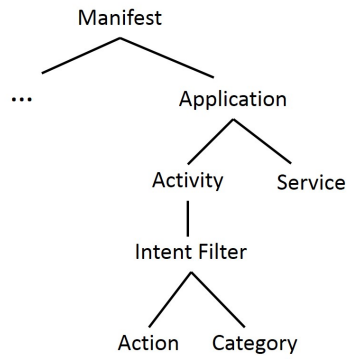

Fig. 2. Abstract Syntax Tree for manifest in SmartGen

3 Proposed System

In RUGRAT, random program generation is done basis custom constraints. These constraints are taken as user input from a desktop application or can be loaded using an XML file. The tool then generates number of Java files inside a package as mentioned by the user with equal Lines of Code in the individual class files. The goal of the tool is to create random programs that can be used as bench-marked codes to let experimenters to quickly generate large number of random codes as per their criteria.

XML UI generation for android.

3.1 Extending RUGRAT with Android code capability

This system basically extends RUGRAT to add Android capability in it. The existing tool used symbolic tree parsing to randomly generate Java codes in a package that can be used for bench-marking.

As shown in Figure 1, RUGRAT uses AST for generating the basic structures of random program generation. It starts with programs at the root and progresses through the tree

as shown till non-terminal nodes. Terminals are chosen only if non-terminals reach the LoC as depicted by user input. SmartGen uses this functionality of RUGRAT as basis for developing random Android codes.

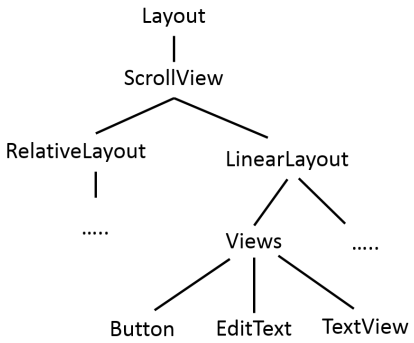

Fig. 3. Abstract Syntax Tree for UI in SmartGen

SmartGen uses same approach, additionally uses XML and Manifest Generation as shown in Figure 2. The AST starts with a static manifest generation which takes number of activities as user input (custom constraint). It typically comprises of Java files, XML tags and UI elements.

The UI Elements that are to be displayed on the application screen are a part of XML file that is generated as part of SmartGen's execution.

On the code side of RUGRAT, the tool typically splits the total LoC into equal size java files. Also, number of variables and methods is equal. In the files that generates loops (for, if) generate fixed LoC in loop body. Moreover, The tool prefers Logical AND operator (75%)than the OR (25%) operator [2]. SmartGen along with other random paramters, randomizes aforesaid parameters as well.

3.2 Android methods and classes

SmartGen generates XML tags which are most commonly used in Android applications. These are related to the UI part of the random code generation. Moreover, in Android, one can have Java classes the way other Java applications can have; however, they are not directly linked with any UI elements. Instead, Android uses the concept of Activities which is the most vital part of Android application life-cycle. Each of the activity can be viewed as single, focusing thing a user can do. [1.] Bundles are then used to pass data between various Android activities.

Below are key Android methods, classes and attributes which are used in the generated code. [8]

- Android XML tags
  - LinearLayout
    In Android, the display is typically viewed as a grid with number of rows and columns. LinearLayout displays all layout element in aforesaid rows or columns.
  - TextView
    This tag displays aforesaid view in text.
  - Button
    Displays an button control element on the screen.
  - EditText
    Allows the user to enter input using soft keyboard or any attached input source.
- Android Java classes
  - Bundle
    As mentioned earlier, in order to pass data from one activity or class to other, Android uses bundle to store activity data that could be used to recreate activity or transfer to other activity.
- Android Java methods
  - onCreate()
    On activity creation, the activity enters the Created state. In the onCreate() method, basic application startup logic that should happen only once for the entire life of the activity are performed. For example, an implementation of onCreate() might bind data to lists, initialize background threads, and instantiate some class-scope variables. This method receives the parameter savedInstanceState, which is a Bundle object containing the activity's previously saved state. If the activity has never existed before, the value of the Bundle object is null. [8]
  - onResume()
    When the activity enters the Resumed state, it comes to the foreground from the paused state, and then the system invokes the onResume() callback. This is the state in which the app interacts with the user. The app stays in this state until something happens to take focus away from the app. Such an event might be, for instance, receiving a phone call, the users navigating to another activity, or the device screens turning off or minimizing the app. [8]
  - onStart()
    When the activity enters the Started state, the system invokes this callback. The onStart() call makes the activity visible to the user, as the app prepares for the activity to enter the foreground and become interactive. For example, this method is where the app initializes the code that maintains the UI. It might also register a BroadcastReceiver that monitors changes that are reflected in the UI. [8]
  - onPause()
    The system calls this method as the first indication that the user is leaving the activity (though it does not always mean the activity is being destroyed). The onPause() method is used to pause operations such animations and music playback that should not continue while the Activity is in the Paused state, and that is expected to resume shortly. [8]

The limitations of RUGRAT have been taken into consideration while designing and developing SmartGen. Along with other loop statement - if, if-else and while, it also

generates random Android benchmark applications with do-while and switch statement. Java Generic methods and classes are not supported in RUGRAT; however, they are supported in SmartGen. UIElements class uses them. SmartGen generates a package with Java files, Android files, XML for UI elements and Manifest XML as part of an application.

4 Experiments

In order to validate the applications generated by SmartGen, various RAT tools were used. The questions to address were to see how similar are the SmartGen applications to third party generated Android apps. FindBugs and JLint were also used to find out and report bugs in the applications generated by SmartGen.

4.1 Setup

The setup used to develop this tool is - Linux OS with Java 1.8, IntelliJ IDEA, Android studio to test the auto-generated code. The tool however, does not depend on this configuration as minimum configuration. It has been tested for backward compatibility. Android API 19 onwards can run programs generated by this tool.

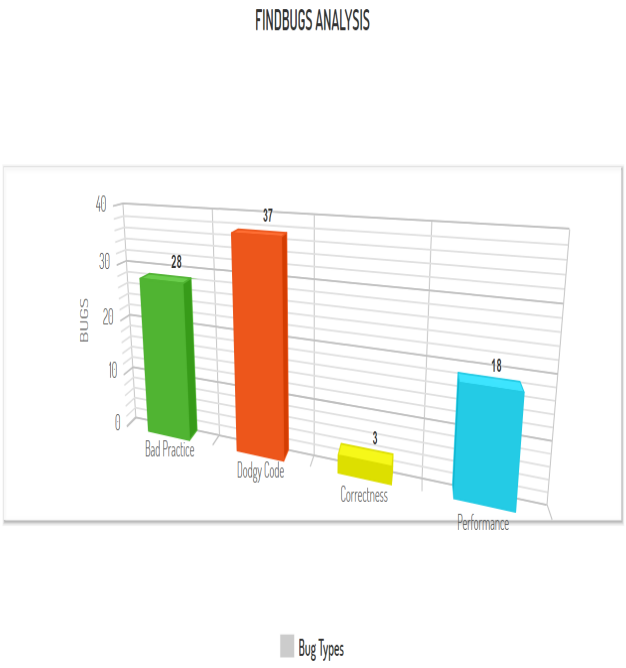

Fig. 4. FindBugs analysis on SmartGen's result

The above graph demonstrates the result of analysis over SmartGen's output file using FindBugs. The code generated is imported in Android Studio and is then compiled. After the compilation, FindBugs is used to analyze all files in the project.

After the analysis, the results were segregated into 4 bug types viz., Bad Practice, Dodgy Code, Correctness, Performance.

- Correctness bug
  Probable bug - an apparent coding mistake resulting in code that was probably not what the developer intended. We strive for a low false positive rate.
- Bad Practice
  Violations of recommended and essential coding practice. Examples include hash code and equals problems, clone-able idiom, dropped exceptions, realizable problems, and misuse of finalize. We strive to make this analysis accurate, although some groups may not care about some of the bad practices.
- Dodgy Code
  Code that is confusing, anomalous, or written in a way that leads itself to errors. Examples include dead local stores, switch fall through, unconfirmed casts, and redundant null check of value known to be null. More false positives accepted. In previous versions of FindBugs, this category was known as Style.

Performance:
Code that further can be optimized using some coding techniques. [9] The results of the analysis were as follows:

- Bad Practice: 28
- Dodgy Code: 37
- Correctness: 3
- Performance: 18

After evaluating the results, SmartGen code generation techniques can be further improved.

5 Future Work

The tool does not generate the directory structure as needed by the Android studio, which can be made so in time to come. Also, the programs generated are single threaded as of now, which can be multi-threaded in future. The randomly generated applications have a limit on Android classes in order to avoid complications during run time (increased LoC). Moreover, the generated applications only have integer variables and methods, which can be worked on in future.

6 Conclusion

This paper developed a scheme to automatically generate random Android applications which can be used as benchmarks for program analysis. This approach made use of Abstract Syntax Trees to generate Manifest file, UI XML. Java files were generated using the RUGRAT code which uses stochastic parse trees.

References

[1] Wikipedia, "Program analysis."
[2] I. Hussain, C. Csallner, M. Grechanik, C. Fu, Q. Xie, S. Park, K. Taneja, and B. Hossain, "Evaluating program analysis and testing tools with the rugrat random benchmark application generator," in *Proceedings of the Ninth International Workshop on Dynamic Analysis*, pp. 1–6, ACM, 2012.
[3] S. M. Blackburn, R. Garner, C. Hoffmann, A. M. Khang, K. S. McKinley, R. Bentzur, A. Diwan, D. Feinberg, D. Frampton, S. Z. Guyer, *et al.*, "The dacapo benchmarks: Java benchmarking development and analysis," in *ACM Sigplan Notices*, vol. 41, pp. 169–190, ACM, 2006.
[4] D. R. Slutz, "Massive stochastic testing of sql," in *VLDB*, vol. 98, pp. 618–622, 1998.
[5] M. Grechanik, C. McMillan, L. DeFerrari, M. Comi, S. Crespi, D. Poshyvanyk, C. Fu, Q. Xie, and C. Ghezzi, "An empirical investigation into a large-scale java open source code repository," in *Proceedings of the 2010 ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*, p. 11, ACM, 2010.

[6] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in c compilers," in *ACM SIGPLAN Notices*, vol. 46, pp. 283–294, ACM, 2011.

[7] B. Daniel, D. Dig, K. Garcia, and D. Marinov, "Automated testing of refactoring engines," in *Proceedings of the the 6th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering*, pp. 185–194, ACM, 2007.

[8] A. O. S. Project, "Introduction to android api."

[9] U. of Maryland, "Findbugs 1.2 demo and results."

[10] A. V. Aho, R. Sethi, and J. D. Ullman, *Compilers, Principles, Techniques*. Addison wesley Boston, 1986.

[11] S. M. Blackburn, K. S. McKinley, R. Garner, C. Hoffmann, A. M. Khan, R. Bentzur, A. Diwan, D. Feinberg, D. Frampton, S. Z. Guyer, *et al.*, "Wake up and smell the coffee: evaluation methodology for the 21st century," *Communications of the ACM*, vol. 51, no. 8, pp. 83–89, 2008.

[12] S. Cohen and B. Kimelfeld, "Querying parse trees of stochastic context-free grammars," in *Proceedings of the 13th International Conference on Database Theory*, pp. 62–75, ACM, 2010.

[13] C. Collberg, G. Myles, and M. Stepp, "An empirical study of java bytecode programs," *Software-Practice and Experience*, vol. 37, no. 6, pp. 581–642, 2007.

[14] P. Cuoq, B. Monate, A. Pacalet, V. Prevosto, J. Regehr, B. Yakobowski, and X. Yang, "Testing static analyzers with randomly generated programs," in *NASA Formal Methods Symposium*, pp. 120–125, Springer, 2012.

[15] H. Do, S. Elbaum, and G. Rothermel, "Supporting controlled experimentation with testing techniques: An infrastructure and its potential impact," *Empirical Software Engineering*, vol. 10, no. 4, pp. 405–435, 2005.

[16] B. Dufour, K. Driesen, L. Hendren, and C. Verbrugge, "Dynamic metrics for java," in *ACM SIGPLAN Notices*, vol. 38, pp. 149–168, ACM, 2003.

[17] P. Godefroid, A. Kiezun, and M. Y. Levin, "Grammar-based whitebox fuzzing," in *ACM Sigplan Notices*, vol. 43, pp. 206–215, ACM, 2008.

[18] K. V. Hanford, "Automatic generation of test cases," *IBM Systems Journal*, vol. 9, no. 4, pp. 242–257, 1970.

[19] A. Joshi, L. Eeckhout, R. H. Bell Jr, and L. K. John, "Distilling the essence of proprietary workloads into miniature benchmarks," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 5, no. 2, p. 10, 2008.

[20] K. Kanoun and L. Spainhower, *Dependability benchmarking for computer systems*, vol. 72. John Wiley & Sons, 2008.

[21] R. Lämmel and W. Schulte, "Controllable combinatorial coverage in grammar-based testing," in *IFIP International Conference on Testing of Communicating Systems*, pp. 19–38, Springer, 2006.

[22] J. R. Levine, "Linkers & loaders. 1999," 1999.

[23] R. Majumdar and R.-G. Xu, "Directed test generation using symbolic grammars," in *Proceedings of the twenty-second IEEE/ACM international conference on Automated software engineering*, pp. 134–143, ACM, 2007.

[24] P. M. Maurer, "Generating test data with enhanced context-free grammars," *Ieee Software*, vol. 7, no. 4, pp. 50–55, 1990.

[25] G. McDaniel, "Ibm dictionary of computing. 1994."

[26] K. S. Nash, "Information technology budgets: Which industry spends the most," *URL: http://tinyurl.com/InformationTechnologyBudgets), N ov*, 2007.

[27] P. Purdom, "A sentence generator for testing parsers," *BIT Numerical Mathematics*, vol. 12, no. 3, pp. 366–375, 1972.

[28] N. Rutar, C. B. Almazan, and J. S. Foster, "A comparison of bug finding tools for java," in *Software Reliability Engineering, 2004. ISSRE 2004. 15th International Symposium on*, pp. 245–256, IEEE, 2004.

[29] R. H. Saavedra and A. J. Smith, "Analysis of benchmark characteristics and benchmark performance prediction," *ACM Transactions on Computer Systems (TOCS)*, vol. 14, no. 4, pp. 344–384, 1996.

[30] M. Schwab, M. Karrenbach, and J. Claerbout, "Making scientific computations reproducible," *Computing in Science & Engineering*, vol. 2, no. 6, pp. 61–67, 2000.

[31] E. G. Sirer and B. N. Bershad, "Using production grammars in software testing," in *ACM SIGPLAN Notices*, vol. 35, pp. 1–13, ACM, 1999.

[32] J. W. A. Ward, "Role of application benchmarks in the dod hpc acquisition process," *US Army Engineer Research and Development Center, ERDC MSRC Resource*, 2005.

[33] T. Yoshikawa, K. Shimura, and T. Ozawa, "Random program generator for java jit compiler test system," in *Quality Software, 2003. Proceedings. Third International Conference on*, pp. 20–23, IEEE, 2003.

[34] H. Zhang and H. B. K. Tan, "An empirical study of class sizes for large java systems," in *Software Engineering Conference, 2007. APSEC 2007. 14th Asia-Pacific*, pp. 230–237, IEEE, 2007.